İÇİNDEKİLER

- Problem Karmaşıklığı-> Asal Sayı Bulma, Maksimum AltDizi Problemi (Max. Subarray Problem)
- Data Yapıları & Sıralama Algoritmaları (Sorting Algorithms)
- Arama Algoritmaları-> Tam arama (Complete Search), İkili Arama (Binary Search), Açgözlü Algoritmalar (Greedy Approach)
- Sorgular(Queries) -> Dinamik Programlama
- Knapsack & Backtracking & Bazı Algoritmalar (Huffman Coding, Harf Sayısı Algoritması(Levenshtein Distance))

- 2.Aşama Örnek Soru «Veyalar Diyarı» 2017 2.aşama sorusu
- En kısa yol algoritmaları
- Minimum spanning tree algorithms
- 2.Aşama Örnek Soru «Zotiland» 2017 2.aşama sorusu
- 2.Aşama Örnek Soru «GrontaBric» 2017 2.aşama sorusu

- 2.Aşama Örnek Soru «Müzik Kutusu» 2017 2.aşama sorusu
- 2.Aşama Örnek Soru «Deterministik Oyun Makinesi» 2017 2.aşama sorusu

Problem Karmaşıklığı

Algoritmanın verimliliği olimpiyatlarda zor soruları çözmek için çok önemlidir. Bir soruyu uzun sürede çözen bir algoritma yerine kısa sürede çözen algoritma tercih edilir. Girdi sayısına göre problemin çözüm zamanını gösterimine «zaman karmaşıklığı» «time complexity» denir.

```
for (int i = 1; i <= n; i++) {
    // code
}
```

O(n)

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        // code
    }
}
```

$O(n^2)$

```
for (int i = 1; i <= 3*n; i++) {
    // code
}
```

```
for (int i = 1; i <= n+5; i++) {
    // code
}
```

```
for (int i = 1; i <= n; i += 2) {
    // code
}
```

O(n)

```
for (int i = 1; i <= n; i++) {
    for (int j = i+1; j <= n; j++) {
        // code
    }
}
```

$O(n^2)$

```
for (int i = 1; i <= n; i++) {
    // code
}
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        // code
    }
}
for (int i = 1; i <= n; i++) {
    // code
}
```

$O(n^2)$

Problem Karmaşıklığı

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        // code
    }
}
```

O(n.m)

```
void f(int n) {
    if (n == 1) return;
    f(n-1);
}
```

O(n)

```
void g(int n) {
    if (n == 1) return;
    g(n-1);
    g(n-1);
}
```

O(2^n)

function call	number of calls
$g(n)$	1
$g(n-1)$	2
$g(n-2)$	4
$\cdots$	$\cdots$
$g(1)$	2^{n-1}

$$1+2+4+\cdots+2^{n-1}=2^n-1=O(2^n).$$

Asal Sayılar Algoritması

Girilen bir sayı asalsa ekrana 1, değilse 0 yazan bir algoritma yazınız.

O(n)

O(√n) yapabilir miyiz?

Maksimum AltDizi Problemi

−1	2	4	−3	5	2	−5	2

- İçinde negatif sayıların olduğu *n* elemanlı bir dizi olsun
- Toplamları maksimum olan ardışık elemanlardan oluşan altdiziyi bulunuz
- Boş dizinin toplamı 0'dır.

−1	2	4	−3	5	2	−5	2

- Cevap :10

```
#include <iostream>

using namespace std;

int main()
{
    int array[8]= {-1,2,4,-3,5,2,-5,2};
    int a,b,best;
    int n=8;

    for (int a=0;a <n ; a++) {
     for (int b=a;b <n ; b++) {
         int sum=0;
         for (int k=a;k<=b;k++) {
             sum += array[k];

         }
         best = max(best,sum) ;
     }
    }
  printf("%d",best);
}
```

$O(n^3)$

Maksimum AltDizi Problemi

−1	2	4	−3	5	2	−5	2

- İçinde negatif sayıların olduğu *n* elemanlı bir dizi olsun
- Toplamları maksimum olan ardışık elemanlardan oluşan altdiziyi bulunuz
- Boş dizinin toplamı 0'dır.

−1	2	4	−3	5	2	−5	2

- Cevap :10

```
#include <iostream>

using namespace std;

int main()
{
    {
   int array[8]= {-1,2,4,-3,5,2,-5,2};
   int a,b,best;
   int n=8;
   best =0;
   for (int a=0;a <n ; a++) {
    int sum=0;
    for (int b=a;b <n ; b++) {
            sum += array[b];
            best = max(best,sum) ;
    }
   }
  printf("%d",best);
}
}
```

$O(n^2)$

Maksimum AltDizi Problemi

−1	2	4	−3	5	2	−5	2

- İçinde negatif sayıların olduğu *n* elemanlı bir dizi olsun
- Toplamları maksimum olan ardışık elemanlardan oluşan altdiziyi bulunuz
- Boş dizinin toplamı 0'dır.

−1	2	4	−3	5	2	−5	2

- Cevap :10

array size n	Algorithm 1	Algorithm 2	Algorithm 3
10^2	0.0 s	0.0 s	0.0 s
10^3	0.1 s	0.0 s	0.0 s
10^4	> 10.0 s	0.1 s	0.0 s
10^5	> 10.0 s	5.3 s	0.0 s
10^6	> 10.0 s	> 10.0 s	0.0 s
10^7	> 10.0 s	> 10.0 s	0.0 s

```
#include <iostream>

using namespace std;

int main()
{
    {
   int array[8]= {-1,2,4,-3,5,2,-5,2};
   int best=0;
   int sum=0;
   int n=8;

   for (int k=0;k <n ; k++) {

            sum = max(array[k], sum+array[k]);
            best = max(best,sum) ;

   }
  printf("%d",best);
}
}
```

O(n)

Ödev

- Verilen n,m değeri için $\binom{n}{m}$ kombinasyonunu yazdıran programı alttaki 2 yöntemle hazırlayınız.

1- Recursive(özyinelemeli fonksiyonla)
2- for döngüsüyle

- Neden Önemli?
  - Bir dizide aynı elemandan 2 tane var mı?
  - Bir dizideki en fazla tekrar eden eleman hangisidir? sorularını sıralama algoritmalarıyla çözebiliriz.

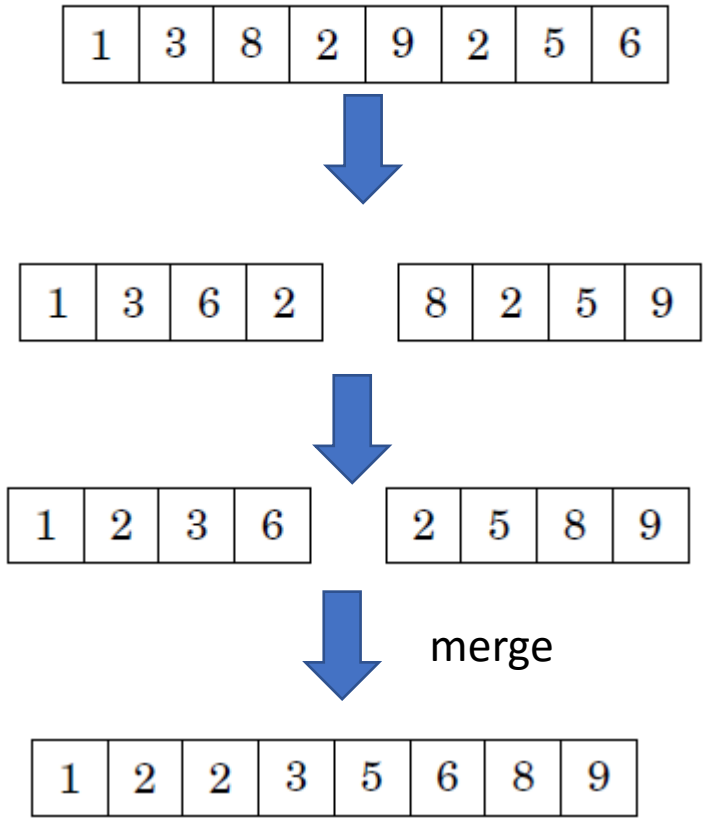
Bubble Sorting $O(n^2)$

1	3	8	2	9	2	5	6

1	2	2	3	5	6	8	9

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n-1; j++) {
        if (array[j] > array[j+1]) {
            swap(array[j],array[j+1]);
        }
    }
}
```

Merge Sorting O(n.log(n))



C++ 'da sıralama

```
#include <iostream>
//sort için alttaki library'i eklemek lazım
#include <algorithm>


using namespace std;

int main()
{
  int n=7;
  int a[]={4,2,5,3,5, 8,3};
  sort(a,a+n);
  for(int i=0;i<n; i++)
    printf("%d ",a[i]);
}
```

- For döngüsündeki n değerini değiştirerek deneme yapalım.

Counting Sort O(n)

Eğer dizideki rakamlarının min ve max değerlerini bilirsem ne olur?

1	3	6	9	9	3	5	9

Yeni bir dizi oluşturalım.

1	2	3	4	5	6	7	8	9
1	0	2	0	1	1	0	0	3

```
#include <iostream>

using namespace std;

int main()
{
    int val;
    int a[]={4,2,5,3,5, 8,3};
    // max değerin 8 olduğunu bildiğim için 8 elemanlık bir dizi yaratıyorum.
    int b[8]={0,0,0,0,0,0,0,0};
    int n = sizeof(a) / sizeof(a[0]);
    for(int i=0;i<n;i++)
    {  val = a[i]-1;
       ++b[val];
    }
    for(int i=0;i<8;i++)
    {if (b[i])
        for(int j=1;j<=b[i];j++)
        printf("%d ",i+1);
    }
}
```

Dinamik diziler
vector

- Dinamik dizi oluşturma
- Listeye eleman ekleme
- Listedeki son elemanı bulma
- Listedeki son elemanı çıkarma
- Listeyi sıralama
- Listeyi tersten sıralama
- Listenin maksimum elemanı ve minimum elemanını bulma

```
int main()
{
   vector<int> v;
   v.push_back(3);
   v.push_back(2);
   v.push_back(5);

   for(int i=0;i<v.size() ;i++)
    cout <<v[i]<<"\n"; // 3 2 5

   // son elemanı çıkarmak için
   v.pop_back();
   // son elemanı bulmak için
   cout << v.back() <<"\n"; //2

   v.push_back(3);
   v.push_back(8);
   v.push_back(6);
   // listeyi sıralamak için
   sort(v.begin(),v.end());
   for(int i=0;i<v.size() ;i++)
   cout <<v[i]<<"\n"; //2 3 3 6 8

   // listeyi tersten sıralamak için
   sort(v.rbegin(),v.rend());
   for(int i=0;i<v.size() ;i++)
   cout <<v[i]<<"\n"; //8 6 3 3 2

   vector<int> d(10); // 10 elemanlık tüm elemanları 0 olan bir dizi
    for(int i=0;i<d.size() ;i++)
   cout <<d[i]<<"\n"; // 0 0 0 0 0 0 0 0 0 0

    vector<int> e(10,5); // 10 elemanlık tüm elemanları 5 olan bir dizi
    for(int i=0;i<e.size() ;i++)
   cout <<e[i]<<"\n"; // 5 5 5 5 5 5 5 5 5 5
```

```
   // listenin maksimum ve minimum değeri
   int maks;
   maks = *max_element(v.begin(), v.end());
     cout <<"Maks ="<< maks;
     cout <<"\nMin ="<< *min_element(v.begin(), v.end());
}
```

Veri Yapıları

Set multiset

- Kümeye eleman ekleme
- Kümede x elemanı var mı?
- Kümeden eleman çıkarma
- Tekrarlı elemanlı kümeler
- Kümenin eleman sayısı

```
int main()
{
  set<int> s;
  s.insert(3);
  s.insert(2);
  s.insert(5);
  s.insert(3);
  //kümedeki bir elemanın kaç tane olduğu
  cout<<s.count(3)<<"\n"; //1
  cout<<s.count(4)<<"\n"; //0
  //kümenin eleman sayısı
  cout<<s.size()<<"\n"; //3

  multiset<int> ms;
  ms.insert(3);
  ms.insert(2);
  ms.insert(5);
  ms.insert(3);
  //kümedeki bir elemanın kaç tane olduğu
  cout<<ms.count(3)<<"\n"; //2
  cout<<ms.count(4)<<"\n"; //0

   //kümenin eleman sayısı
  cout<<ms.size()<<"\n"; //4
  //kümeden eleman çıkarma
  s.erase(4);
  s.erase(2);
  ms.erase(s.find(3));
  cout<<ms.size()<<"\n"; //3
}
```

set

- iterator

```
#include <iostream>
#include<bits/stdc++.h>

using namespace std;

int main()
{
  set<int> s;
  s.insert(3);
  s.insert(2);
  s.insert(5);

  set<int>::iterator it =s.begin();
  //it kümedeki en küçük elemanına pointer koyuyor
  cout<<*it<<"\n";

  for(auto it = s.begin(); it!= s.end();it++)
    cout<<*it<<"\n";

}
```

bitset

```
int main()

{
 bitset<10> s(string("0000110110"))    ;
 cout<<s[2]<<"\n";   //tahmin ?
 cout<<s[4]<<"\n";   //tahmin ?
 cout<<s.count()<<"\n";   //4
}
```

Kompleksliği iyileştirmez ama zamanı ve memory kullanımını iyileştirir.

deque

```
deque<int> d;
d.push_back(5);
d.push_back(2); //   [5,2]
d.push_front(3); //    [3,5,2]
d.pop_back(); //[3,5]
d.pop_front(); //[5]
```

stack

```
stack<int> s;
s.push(3);
s.push(2);
s.push(5);
cout<< s.top(); //5
s.pop();
cout<< s.top(); //2
```

queue

```
queue<int> q;
q.push(3);
q.push(2);
q.push(5);
cout<< q.front(); //3
q.pop();
cout<< q.front(); //2
```

Priority_queue

```
priority_queue<int> p;
p.push(3);
p.push(5);
p.push(7);
p.push(2);
cout<< p.top() << "\n"; //7
p.pop();
cout<< p.top() << "\n"; //5
p.pop();
p.push(6);
cout<< p.top() << "\n\n"; //6
p.pop();
```

Priority_queue (minimum için)

```
priority_queue<int,vector<int>, greater<int> > w;
w.push(3);
w.push(5);
w.push(7);
w.push(2);
cout<< w.top() << "\n"; //2
w.pop();
cout<< w.top() << "\n"; //3
w.pop();
w.push(6);
cout<< w.top() << "\n"; //5
```

Ödev

- Verilen bir sayının asal çarpanlarının kümesini veren bir algoritma yazınız
  - Örneğin girdi 24 ise çıktı {2,2,2,3} olmalıdır.

Ödev

Şef atlara çok düşkündür. Onları yarışırken izlemekten zevk alır. Beklendiği gibi, atlarla dolu bir ahırı var. Hafta sonları arkadaşlarıyla birlikte bu atlardan birkaçının yarışını izlemek için ahırına gider. Aşçı, arkadaşlarının yarıştan zevk almasını istiyor ve bu yüzden yarışın yakın olmasını istiyor. Bu, ancak atlar yetenekleri açısından karşılaştırılabilir ise, yani becerilerindeki fark daha azsa gerçekleşebilir.

Ahırda N tane at var. i atın becerisi bir S[i] tamsayısıyla temsil edilir. Aşçı, becerilerindeki fark minimum olacak şekilde yarış için 2 at seçmelidir. Bu sayede çok ilginç bir yarışa ev sahipliği yapabilecektir. Göreviniz, bunu yapmasına yardımcı olmak ve yarışta 2 at arasında mümkün olan minimum farkı bildirmektir.

Girdi:

N tamsayısını içeren bir satırla başlar.

Sonraki satır, i-inci tamsayının S[i] olduğu N boşlukla ayrılmış tamsayı içerir.

Çıktı:

mümkün olan minimum farkı içeren tek bir satır çıktı alın.

kısıtlamalar:

$1 \le T \le 10$

$2 \le N \le 5000$

$1 \le S[i] \le 1000000000$

Örnek Giriş 1

5

4 9 1 32 13

Örnek Çıktı 1

3

Açıklama

Yarış için beceri 1 ve 4 olan atları seçersek minimum fark elde edilebilir.